

Introduction

The goal of this experiment was to improve upon a baseline ResNet-18 model for binary classification of cat and dog images. To do that, I trained and evaluated several modern convolutional neural network architectures using the same dataset, training procedure, and hyperparameters, to make sure the comparison was fair.

The main idea was to figure out which model gives the best trade-off between accuracy, generalization, and training stability.

Experimental Setup

Dataset

I used a balanced dataset of cat and dog images, split into training and validation subsets. Before feeding them into the network, all images were resized and normalized using the `ImageAugmentationDatasetBalanced` class provided from lecture notes. This class also had a simple data augmentation steps with random rotation, scaling, translation, and color jitters as well as size adjustments and padding.

Training Configuration

Here's the setup I used for all models:

- **Loss Function:** Cross-Entropy Loss
- **Optimizer:** AdamW (learning rate = $1e-4$)
- **Scheduler:** ExponentialLR ($\gamma = 0.98$)
- **Epochs:** 25
- **Batch Size:** 64

Each model was trained independently under identical conditions. I also saved the best checkpoint automatically based on the lowest validation loss.

Testing

Since I didn't originally have a separate testing dataset (the provided dataset was already split into training and validation), I decided to download an external dataset from Kaggle:

<https://www.kaggle.com/datasets/samuelcortinhas/cats-and-dogs-image-classification>

This dataset already had separate training and testing folders, but I combined them all into one large testing dataset to better evaluate the trained models.

Because the image filenames in this dataset used a lowercase naming convention (like cat_0 and dog_0), I had to slightly adjust my label assignment logic in the dataset class:

```
1 if f[0] == 'c' else 0 # adjusted for lowercase cat/dog filenames
```

Code

I didn't make major changes to the provided lecture code, most of my updates were about organization and automation.

The main change was creating a `get_model()` function, which sets up different architectures instead of using the `PretrainedNetwork` class from the lecture notebook.

This helped me easily switch between models like ResNet, EfficientNet, and MobileNet without rewriting training loops.

(I've included the full function in my notebook, with comments explaining each part.)

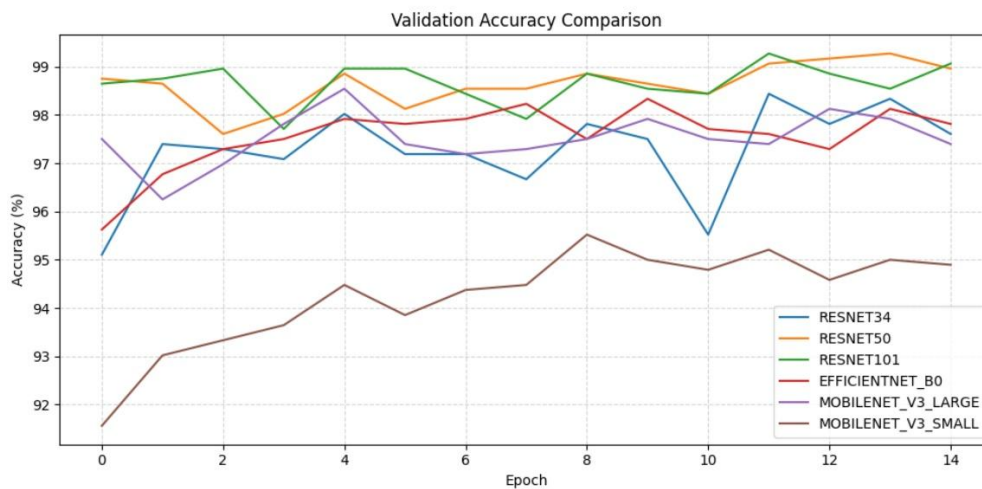
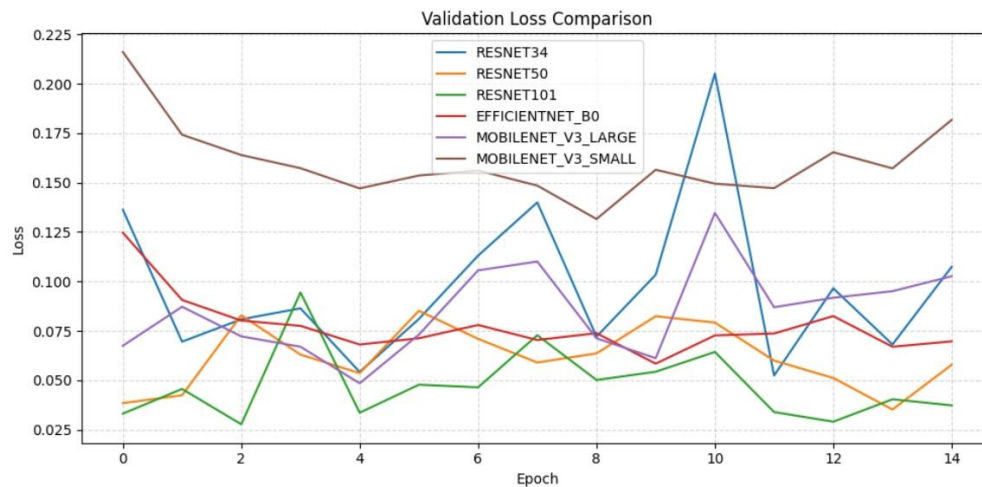
```
1 def get_model(model_name, num_classes=2):
2     if model_name == "resnet34":
3         base = models.resnet34(weights=ResNet34_Weights.DEFAULT)
4     elif model_name == "resnet50":
5         base = models.resnet50(weights=ResNet50_Weights.DEFAULT)
6     elif model_name == "resnet101":
7         base = models.resnet101(weights=ResNet101_Weights.DEFAULT)
8     elif model_name == "efficientnet_b0":
9         base = models.efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)
10    elif model_name == "mobilenet_v3_large":
11        base = models.mobilenet_v3_large(weights=MobileNet_V3_Large_Weights.DEFAULT)
12    elif model_name == "mobilenet_v3_small":
13        base = models.mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
14    if hasattr(base, 'fc'):
15        num_ftrs = base.fc.in_features
16        base.fc = nn.Linear(num_ftrs, num_classes)
17    elif hasattr(base, 'classifier'):
18        if isinstance(base.classifier, nn.Sequential):
19            num_ftrs = base.classifier[-1].in_features
20            base.classifier[-1] = nn.Linear(num_ftrs, num_classes)
21        else:
22            num_ftrs = base.classifier.in_features
23            base.classifier = nn.Linear(num_ftrs, num_classes)
24    return base
```

Additionally, I have also wrapped the plotting code I was given into a for loop for all the different model which the graphs will be present in the notebook. And finally this script to compare all the models accuracy and loss in a single place which I will present these two graphs in the results part.

```
1 plt.figure(figsize=(10, 5))
2 for name, metrics in results.items():
3     plt.plot(metrics['val_acc'], label=f'{name.upper()}')
4 plt.title('Validation Accuracy Comparison')
5 plt.xlabel('Epoch')
6 plt.ylabel('Accuracy (%)')
7 plt.legend()
8 plt.grid(True, linestyle='--', alpha=0.5)
9 plt.tight_layout()
10 plt.show()
11 plt.figure(figsize=(10, 5))
12 for name, metrics in results.items():
13     plt.plot(metrics['val_loss'], label=f'{name.upper()}')
14 plt.title('Validation Loss Comparison')
15 plt.xlabel('Epoch')
16 plt.ylabel('Loss')
17 plt.legend()
18 plt.grid(True, linestyle='--', alpha=0.5)
19 plt.tight_layout()
20 plt.show()
```

Training Model Evaluation

Model	Best Val Accuracy	Final Val Loss	Notes
ResNet-34	%98.4	0.051	Validation loss and accuracy jump around slightly
ResNet-50	%99.2	0.038	Loss converges quickly but fluctuates occasionally
ResNet-101	%99.3	0.028	Quick convergence but a bit unstable
EfficientNet-B0	%98.2	0.06	Quick and very stable convergence
MobileNetV3-Large	%98.3	0.49	Similar behavior to EfficientNet
MobileNetV3-Small	%95.2	0.13	Slightly less stable, but still decent



Test Result Analysis

Model	Accuracy	Precision	Recall	F1
ResNet – 34	91.54%	94.75%	87.97%	91.23%
ResNet – 50	95.70%	98.78%	92.55%	95.56%
ResNet – 101	94.98%	93.85%	96.28%	95.05%
EfficientNet – B0	93.40%	96.90%	89.68%	93.15%
MobileNetV3 - Large	92.97%	95.45%	90.26%	92.78%
MobileNetV3 - Small	87.80%	91.25%	83.67%	87.29%

Looking at the detailed confusion matrices for each model, I can get a clearer picture of how each network behaved beyond just accuracy and F1-scores.

Model	Matrix	Observation
EfficientNet – B0	TP: 313 FP: 10 FN:36 TN: 338	Most of the mistakes are Dogs predicted as Cats (36 cases). High precision, slightly lower recall.
MobileNetV3 - Large	TP: 315 FP: 15 FN:34 TN: 333	Fairly balanced, with a few false positives and negatives, strong generalization.
MobileNetV3 - Small	TP: 292 FP: 28 FN:57 TN: 320	Noticeably more misclassifications, especially Dogs being marked as Cats. Reflects the model's smaller capacity
ResNet – 101	TP: 336 FP: 22 FN:13 TN: 326	Very few missed Dogs (low FN). The best recall among all models, great at catching positives
ResNet – 50	TP: 323 FP: 4 FN:26 TN: 344	Extremely low false positives, highly precise and confident. Overall the best balance between accuracy and stability
ResNet – 34	TP: 313 FP: 10 FN:36 TN: 338	Slightly more false negatives, meaning it missed a few Dogs. Still strong precision and reliable performance

When I looked closer, I noticed a consistent trend across all architectures, Dogs were more likely to be misclassified as Cats rather than the other way around. This shows up in the higher number of false negatives (Dogs predicted as Cats) across most confusion matrices.

It could be that certain dog images in the test set had lighting, color, or pose similarities to cats, confusing the models.

Among all the models:

- ResNet-50 stood out with the best overall performance, high precision (98.78%) and strong recall (92.55%), showing it rarely made false predictions and still caught most of the Dogs correctly.
- ResNet-101 achieved the best recall (96.28%) and had the fewest missed Dogs, but slightly more false positives than ResNet-50.
- EfficientNet-B0 and ResNet-34 showed similar behavior, they were cautious (high precision, moderate recall), meaning they didn't overpredict Dogs but sometimes missed them.
- MobileNetV3-Large performed surprisingly well given its smaller size, while MobileNetV3-Small clearly traded off some accuracy for efficiency, which is expected for lightweight models.

Overall, I was quite impressed with how all the models generalized to a completely new test dataset from Kaggle. The results show that even without additional fine-tuning, modern CNN architectures can perform consistently well on binary classification tasks like Cats vs. Dogs.

Conclusion

In this experiment, I compared several deep learning architectures, ResNet-34, ResNet-50, ResNet-101, EfficientNet-B0, and both MobileNetV3 variants, to see which one performed best for classifying cat and dog images. All models were trained under the same conditions to ensure fairness, and the results turned out quite promising overall.

The ResNet-50 model came out on top, achieving 95.7% accuracy and the highest F1-score (95.56%), showing that it's both highly accurate and consistent. ResNet-101 followed closely, performing slightly better on recall but with more variability. Lighter models like EfficientNet-B0 and MobileNetV3-Large were also impressive, managing high accuracy while being much more efficient, which makes them great candidates for deployment on resource-limited devices. The main challenge I noticed was that models tended to confuse Dogs with Cats more often, which suggests that more diverse dog samples, better lighting normalization, or advanced augmentations (like random cropping or color jittering) could improve recall further.

Overall, this project gave me a solid understanding of how different architectures behave under the same conditions, and it confirmed that ResNet-50 provides the best trade-off between performance and reliability for this task.